# COM661 Full Stack Strategies and Development

# API Coursework Clinic

## Contents


**1. Dataset Selection ..... 2**

*1.1 Preparing the dataset ..... 2*

*1.2 Dataset Size ..... 2*

*1.3 Use of Random Data ..... 3*

*1.4 Range of Data Types ..... 3*

*1.5 Sub-document Collections ..... 4*

**2. Database Queries and API Endpoints ..... 4**

*2.1 Basic Queries ..... 4*

*2.2 Queries Specific to the Dataset ..... 4*

**3. Implementation Issues ..... 5**

*3.1 Code Structure ..... 5*

*3.2 Authentication and User Management ..... 6*

*3.3 Endpoints to Support Functionality ..... 6*

*3.4 Flexible Endpoints ..... 7*

**4. Testing and Documentation ..... 7**

*4.1 Testing ..... 7*

*4.2 Documentation ..... 8*

**5. The Submission Package ..... 8**

*5.1 The Application ..... 8*

*5.2 Supporting Evidence ..... 9*

# 1. Dataset Selection

## 1.1 Preparing the dataset

Datasets can come from any source and in any format. However, to import the data into MongoDB, it needs to be transformed (if required) to a JSON array where each array element is a is a document to be imported into the database.

Where the dataset is in a form other than JSON, such as CSV, XLS, XML, etc., you will need to use one of the online converter tools to obtain the JSON format.

Take particular care when the dataset is found as separate (but related) files such as (e.g.) a file of businesses, a file of reviews, and a file of users. Often these files are linked by ID fields, so that an entry in the reviews file will contain a business ID identifying the business to which the review belongs, and a user ID indicating the user that provided the review. These ID fields will not be valid MongoDB **`_id`** values and you will need to pre-process the data before it can be used. The best way to approach this is to **mongoimport** the files to the database as separate collections and perform either or both of the following steps.

i) Where possible, you should embed the collections. In this example, it is appropriate to embed reviews within businesses, so the best process is to iterate across the businesses collection and for each business, find all reviews that match the existing business ID, push them into an array, and add the array as a new **`reviews`** field for the business. When complete, the reviews collection can be dropped from the database.

ii) Where embedding is not appropriate, such as with the users collection, you need to update the link between users and reviews. This time, you need to iterate across all of the reviews for each business and for each review, find the ID in the users collection that matches the user ID in the review. You should then add the user **`_id`** to the review as a new field, along with any other user data that you want to display along with the review such as the user's name.

## 1.2 Dataset Size

There is no fixed maximum or minimum number of documents required in the collection, but you should bear in mind the functionality that you will want to demonstrate. Searching and pagination are common operations that you might want to

display, but if all your data can potentially be displayed on the page at once, then there is little point in providing this functionality. A minimum of 100 documents is suggested as a size that provides scope sufficient functionality, but this is not an absolutely fixed minimum. For example, the dataset might be a based on the Olympic Games, but even though there have only been 30 editions of the Summer Games in total, each Games might contain a very large set of data on countries participating, events, medal winners, etc. You should use your judgement as to whether the data you use contains sufficient information to implement all of the functionality that you want to demonstrate.

## 1.3 Use of Random Data

Building your own dataset is absolutely fine and can be a good way of making sure that it has all of the characteristics that you require. Keep in mind that the data does not need to be "real", but it should be "realistic". For example, generate towns and cities by selecting them randomly from a list, rather than generating a random string. Likewise, you can make use of online resources that will generate realistic names, addresses, birthdays, etc.

Another use of random data is to enhance an existing dataset. This might be to add additional documents to a collection that is interesting but contains too few documents, or to add fields to introduces data types that are not currently present. Again, "realistic" data is the aim, so try to introduce random data that does not look out of place with the real information present in the collection.

## 1.4 Range of Data Types

A range of data types in the collection can help with providing a wider range of functionality. For example, numeric data enables range-based queries such as prices or ages above and/or below certain range limits. Again, it can be worthwhile to enhance the range of data types used by investigating randomly generated data as described above.

Documents that contain hyperlinks to online resources are particularly useful, especially if the hyperlink is to an image that describes the data item and can be used later in the front-end of your application. If your dataset does not contain hyperlinks but you intend to add them for the front-end later, then it is OK to provide a dummy URL at this stage.

### 1.5 Sub-document Collections

Having an embedded sub-document collection is essential in allowing you to provide a wider range of functionality. You should aim to provide at least one sub-document collection in your dataset, even if the data for it is not present in your collection and it needs to be added manually. At the most basic level, this might only involve adding a field as an empty list such as **`"reviews" : []`** to each document, but it would be even better to generate random data for at least some sub-document elements.

## 2. Database Queries and API Endpoints

### 2.1 Basic Queries

The standard CRUD (create, retrieve, update, delete) queries are required, and you will be expected to demonstrate at least one of each for documents and sub-documents. **You should treat the database functionality described in the Biz Directory example**, where we can perform the CRUD operations on businesses and reviews identified by **`_id`**, **as the absolute minimum**. There is so much more functionality that can be provided, identified in the following section.

### 2.2 Queries Specific to the Dataset

A much richer set of functionalities can be provided by supporting queries that are directly related to the dataset to which you are providing access. For example, in the case of the Biz Directory, rather than searching for a business by _id, it would be much more useful (and user-friendly) to provide queries by town, or business type, or by multiple fields; for example, "*find all hairdressing businesses in Belfast*" or "*find all restaurants in Newry that cater for vegetarians*". You should consider the fields provided in your dataset and think about whether the field would support a useful element of functionality.

Consider also how your queries can be parameterised to enhance their usefulness. We have already seen how providing **page_number** and **page_size** parameters to the **GET /api/v1.0/businesses** endpoint can control the subset of businesses that is returned, but we could use a similar approach with any field. For example, a collection of cars in a car dealer application might support a query where the user provides a minimum and maximum price that they will consider, or a maximum mileage or earliest year acceptable for a used car.

The Aggregation Pipeline is a very powerful tool for specifying complex MongoDB queries as multi-stage processes and you should try to include at least one example of this. It can be particularly powerful when the pipeline is constructed dynamically according to values provided by the user. For example, if an endpoint URL contains a list of parameters which are optional, then the Pipeline can be built according to which parameters have been supplied in the request.

Geolocation data is a particularly good source of complex functionality and where your dataset contains this, you should definitely consider exploiting it. If your dataset does not contain coordinate information but does have a location field (such as a town), then you should consider adding randomly generated (but realistic) coordinate data as we did with the Biz Directory in Practical BE07 (Section 7.3.1). This might be used in a variety of ways; for example, when retrieving a business for display it might be useful to support an endpoint that retrieves similar businesses in the neighbourhood.

# 3. Implementation Issues

## 3.1 Code Structure

It is important that your code is readable (and therefore maintainable). This can be achieved in several ways.

i) You could consider using Flask Blueprints to modularise your code into separate units, with each unit registered with and imported into the main application file. This is not compulsory (as the main benefit of Blueprints is the ability to share them between applications, which does not apply in your use-case), but it certainly promotes good code structure.

ii) If your code is contained in a single source file (e.g., app.py), you should make sure that the structure of the file supports readability. Grouping related endpoints together (such as all business endpoints, followed by all reviews endpoints, followed by all authentication endpoints) can help with this and you should clearly identify the boundaries of each group with distinctive comments.

iii) Appropriate variable and function names should be used. The worst submission is one that takes the Biz Directory code, changes the database, collection, and field names but leaves everything else as it originally was. In this case, an endpoint to retrieve all (e.g.) hotels remains as **GET**

**api/v1.0/businesses** implemented by a function called **`show_all_businesses()`**! Ideally, you should not use the Biz Directory as a starting point, but should begin from scratch, writing each endpoint as it is required and choosing variable and function names appropriate to your application.

## 3.2 Authentication and User Management

The Biz Directory application demonstrates three levels of access – endpoints that are accessible by anyone, endpoints that are accessible only by registered and logged-in users, and endpoints that are accessible only by registers and logged-in users with "admin" privilege. This is implemented by decorators that test the availability and content of a JSON Web Token that is created when the user logs in and which is added to a blacklist when the user logs out.

This is a good model to follow, making sure that you provide appropriate access rights to the endpoints in your API. For example, in the Biz Directory the ability to edit or delete a review of a business could be reserved not only for "admin" users, but also for the user that originally contributed the review.

It is good practice to record the user that performed each operation that changes the database, for example, "Business 100 edited by Adrian on 19th October 2026", or "Review 200 deleted by Jenny on 20th October 2026". This information could be stored in a new database collection and can provide an additional source of functionality where you could provide "admin" endpoints to track the most active users, businesses/reviews with the most edits, and so on.

## 3.3 Endpoints to Support Functionality

Most API endpoints will be user-facing in the sense that they will correspond to functionality options made available to users at the front-end. However, it can be useful to provide additional endpoints that support functionality rather than users. For example, when implementing pagination, it might be useful to report the number of pages available as well as the page that is currently being displayed. An endpoint **GET /api/v1.0/numberOfPages** might return this value. Similarly, an endpoint that returns the number of businesses in each town, or the number of businesses of each type might be useful in enhancing the usability of the interface.

Sometimes, the need for such endpoints does not become apparent until you are building the API. You can certainly modify the API later to support your front-end

development, but as only the front-end will be marked in CW2 it is better to gain credit for any "supporting" endpoints now by anticipating the need for them. In short – think about the potential front-end while you are developing the back-end.

### 3.4 Flexible Endpoints

The Biz Directory example provides endpoints that allow businesses and reviews to be edited. However, it requires that all fields are provided and if any are missing, the edit operation does not proceed.

It would be much more flexible to permit only the fields that are to be changed to be specified in the request. In this way, if only a business rating is provided (so the name and town are not present), then only the rating is updated, and the other fields remain with their current values.

## 4. Testing and Documentation

### 4.1 Testing

The short-sighted view of testing is that the application should behave properly when the correct input is provided. We have seen how we can check this in Postman by providing  a suite of connected tests that demonstrate that (e.g.) a new business can be added, it can be retrieved, then edited and finally deleted – with the appropriate HTTP status code returned each time demonstrating that the code has followed the appropriate path.

However, a more long-sighted view is that the program should also behave appropriately when the wrong (or no) input is provided. All endpoints should have tests that check all exit routes and all potential paths through the code. For example, in the Biz Directory, when we add a new business there are a few potential scenarios.

i) All required information is provided, and the business is added.

ii) An appropriate JSON Web Token is not present in the request header, so the request is denied.

iii) The token is present but one or more of the compulsory field values is not provided, so the request is denied.

All of these scenarios should be checked in the suite of Postman tests that are implemented.

## 4.2 Documentation

The Postman tool provides a very comprehensive facility that manages the creation and maintenance of documentation for all aspects of the API. You should make use of this to provide documentation for all endpoints, all endpoint parameters, as well as the collection of endpoints as a whole.

Each endpoint should be documented by examples. Consider providing one example for each potential exit point if it provides additional information for the user. Use the documentation of other APIs that you have seen (e.g. OpenWeatherMap) as a guide for the level of detail that can be provided.

Remember that most APIs are developed as services that can be consumed by unknown third parties. As such, it is important that the documentation clearly describes what is possible with the API, how services are requested, and how to interpret the data returned.

You will need to provide your API documentation ass part of the submission package (see the next section), and the easiest way to do this is to publish your Postman documentation on the Web, so please consider this as an option.

# 5. The Submission Package

There are 5 elements to the submission package as described in the Assessment Specification.

## 5.1 The Application

You are required to submit a Zip file containing all files required to run the application. This might be a single ***app.py*** file or could be a group of Blueprint files with the associated main application file. Remember to also include any data files that are needed, but which are not part of the MongoDB database (for example configuration files, etc.).

You should also submit your database files as a separate JSON file for each collection. These should be generated by running **mongoexport** from your database, rather than the original dataset that you have sourced, so that any modifications that you have made are included.

## 5.2 Supporting Evidence

There is a set of supporting evidence that should be included in the submission.

i) A video of no more than 5 minutes duration that shows a demonstration (using Postman) of the API functionality, plus a walkthrough of your code. You do not need to appear in vision on the video, but it must have a voiceover commentary. Please use the video to draw attention to any elements of the submission of which you are particularly proud. The video can be generated by Panopto (built in to the Blackboard platform) or by any other package as long as it can be uploaded in mp4 format.

ii) The Postman-generated documentation for the application should be submitted. This can be in the form of screenshots or an export of the material from Postman but is much more easily done by providing the URL of the documentation published on the Web.

iii) A screenshot of the running of a connected set of Postman tests should be provided. The screenshot should identify the purpose of each test (achieved by giving the tests meaningful names) as well as showing the pass/fail status of each. The screenshot should be presented as a PDF file.

iv) You should provide a Self-Evaluation Report by assessing your application in the spreadsheet provided and including it with your submission package. For each of the marking criteria (each row of the spreadsheet), award yourself a mark out of 10 in the appropriate column (only one column per row should be completed) and provide a short comment justifying the mark. The purpose of this is to help you identify elements that you may have missed and the information you provide has no bearing on your eventual mark (although submitting the Report contributes to the “Submission Package” element of the marking scheme.

v) If you have used AI in the generation of your software, you must include a Design and Implementation document that identifies the elements that have been AI-generated. You should also include detail in the submission (either in this document or in the video commentary) that demonstrates that you understand the code that has been produced.

GOOD LUCK WITH YOUR SUBMISSIONS!